Архитектура высоконагруженных систем

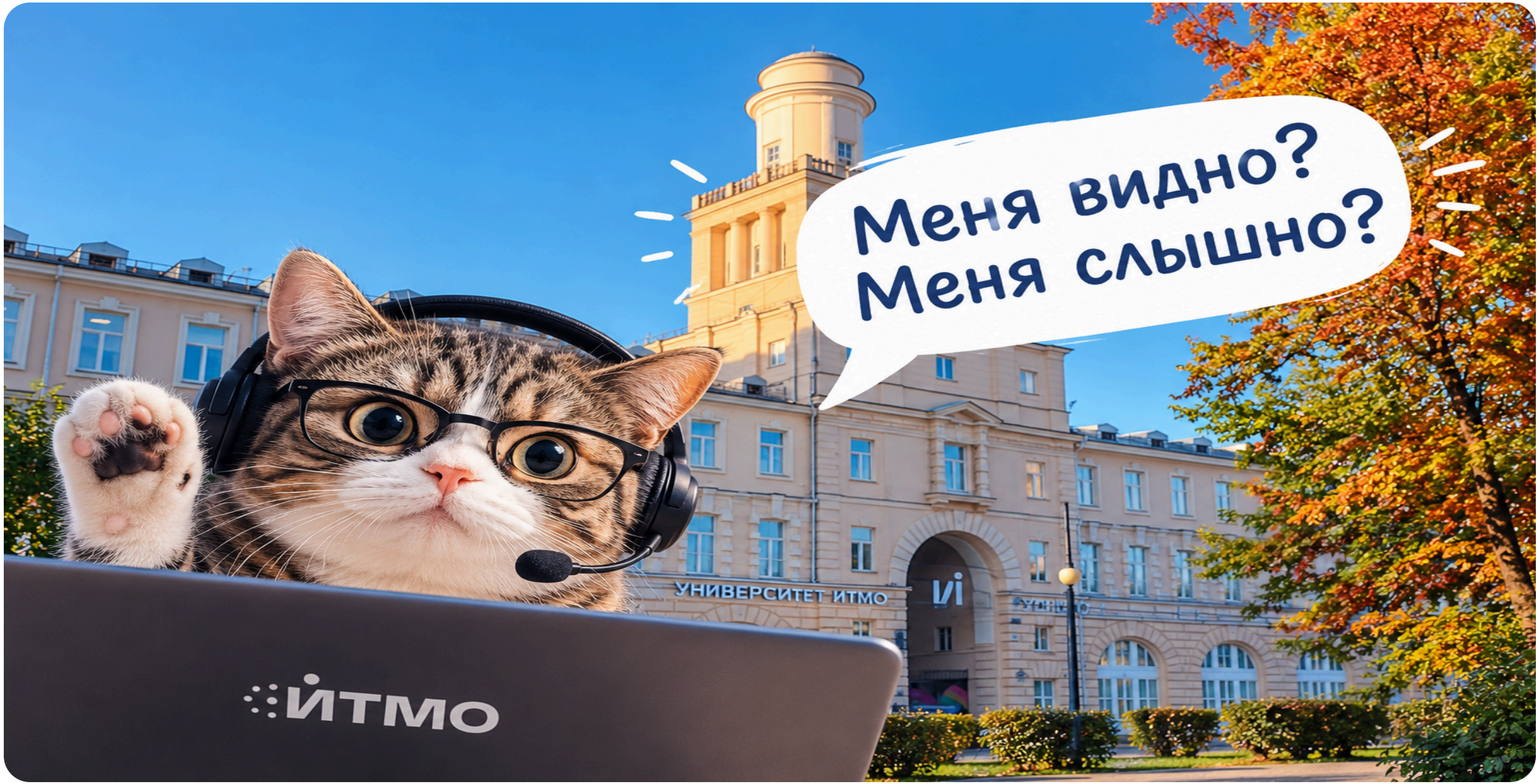
Семён Мартынов, Кирилл Дикалин, Георгий Семёнов

ИНСТИТУТ ПРИКЛАДНЫХ КОМПЬЮТЕРНЫХ НАУК

Национальный исследовательский университет ИТМО

29 октября 2026

Проверка связи



Цели лекции

1. **Выбирать** алгоритм балансировки и **настраивать** проверки состояния, понимая цену L4/L7, sticky sessions и повторов запросов.
2. **Считать** эффект кэша через hit ratio и **выбирать** шаблон работы с ним: cache-aside, read-through, write-through, write-behind.
3. **Объяснять**, откуда берутся устаревшие данные и cache stampede, и **применять** защиту: single-flight, stale-while-revalidate, jitter, раннее обновление.
4. **Проектировать** асинхронный путь запроса: очередь, идемпотентность, пакетирование, объединение запросов, и **оценивать** цену ожидания в очереди.
5. **Выбирать** ключ секционирования, **находить** горячие разделы, **определять**, когда шардирование преждевременно, и **узнавать** архитектурные анти-паттерны.

Вопрос лекции:

какой механизм снимает именно это ограничение и чем мы за него платим?

План лекции

1. Балансировка нагрузки: L4 и L7, алгоритмы, проверки состояния, sticky sessions
2. Кэширование: размещение, шаблоны, TTL и инвалидация, cache stampede
3. Очереди и асинхронность: асинхронный путь, сглаживание пиков, пакетирование, объединение запросов
4. Разделение данных: ключи, перебалансировка, горячие разделы, когда шардирование преждевременно
5. Архитектурные анти-паттерны: цена каждого компонента и выбор механизма по ограничению

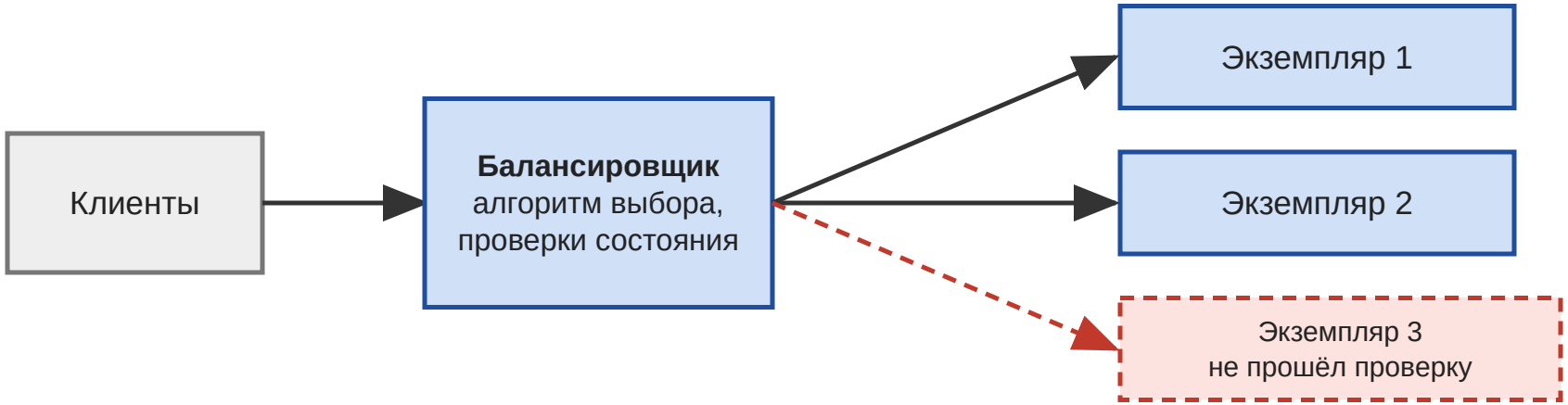
Загадка

Система: **10 000 RPS** на чтение, перед БД стоит кэш с hit ratio **95%**. В обычное время БД получает $0.05 \cdot 10\,000 = 500$ RPS, а её предел **2 000** RPS.

Ночью кэш перезапустили. Через минуту БД на 100% CPU, запросы падают по таймаутам, клиенты их повторяют. Кэш давно снова работает, но система **не возвращается** в норму.

> Кэш ускорял систему. Почему именно он стал причиной простоя?
> Разгадка в разделе 5.2, продолжение в лекции 7.

5.1 Зачем нужен балансировщик



красный: исключён из ротации, трафик не получает

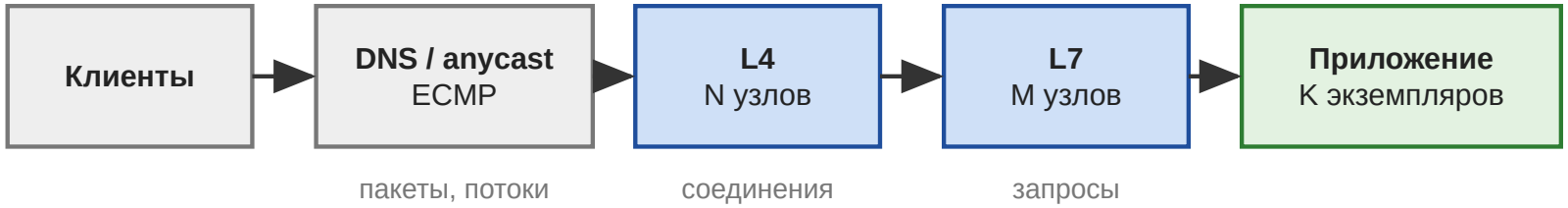
- Балансировщик **распределяет** запросы, **исключает** нездоровые экземпляры и **скрывает** состав парка: экземпляр добавляют или убирают без смены адреса для клиента
- Это условие горизонтального масштабирования из лекции 3: экземпляры без состояния и единая точка входа
- Цена: ещё один сетевой переход (лекция 4) и ещё один компонент, который сам может стать узким местом или точкой отказа

5.1 L4 и L7

	L4 (транспортный уровень)	L7 (прикладной уровень)
Что видит	IP-адреса, порты, TCP/UDP-соединение	метод, URL, заголовки, cookie, метод gRPC
Единица балансировки	**соединение**	**запрос**
Что умеет	раздавать соединения, хэш по адресам и портам	маршрутизация по пути и заголовкам, повторы, TLS-терминация, лимиты, метрики по запросам
Цена	мала: протокол не разбирается	разбор протокола, буферизация, TLS: CPU и память на каждый запрос
Примеры	Linux IPVS, nginx `stream`, Maglev, AWS NLB	nginx `http`, HAProxy (режим http), Envoy, AWS ALB

- Следствие: HTTP/2 и gRPC передают **много запросов в одном соединении**, а L4 выбирает узел один раз на соединение. Все запросы такого соединения идут на один узел, поэтому для равномерности нужна L7-балансировка или балансировка на стороне клиента (лекция 3: долгоживущие соединения)

5.1 Балансировщик тоже масштабируется



чем правее, тем больше работы на единицу трафика (TLS, разбор протокола, правила)

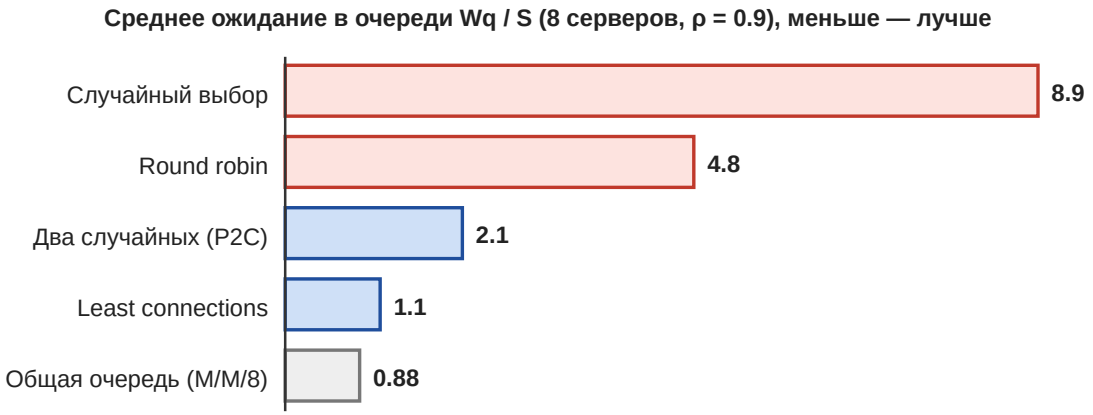
- Парк балансировщиков считается по лекции 3: ёмкость μ, запас, потеря узла. L7 дороже L4 на единицу трафика, поэтому в типовой схеме L4-слой стоит перед L7-слоем
- Maglev (Google): маршрутизаторы раздают пакеты между машинами Maglev (ECMP), каждая выбирает конечный узел; консистентное хэширование и отслеживание соединений уменьшают число разорванных соединений при сбоях (Eisenbud et al., NSDI 2016)
- Один балансировщик без запаса мощности остаётся единой точкой отказа

5.1 Алгоритмы выбора узла

Алгоритм	Как выбирает	Когда подходит	Слабое место
Round robin	по кругу	однородные узлы и запросы	не видит нагрузку: длинные запросы копятся на одном узле
Взвешенный round robin	по кругу, пропорционально весам	узлы разной мощности	веса задаются вручную, реальная загрузка не учитывается
Least connections	узел с наименьшим числом активных соединений или запросов	запросы разной длины	нужны счётчики; новый узел с нулём соединений «притягивает» поток
Хэш по ключу (ip_hash, consistent hash)	один ключ всегда на один узел	локальность: кэш, сессии	перекос при неравномерных ключах
Power of two choices	два случайных узла, берём менее загруженный	много узлов, несколько балансировщиков	не лечит горячий ключ

В nginx по умолчанию используется взвешенный round robin; `ip_hash` берёт первые три октета IPv4-адреса клиента, то есть сеть /24 целиком.

5.1 Сколько стоит выбор алгоритма



- Модель лекции 2: 8 серверов, $\rho = 0.9$, у каждого сервера своя очередь. Для случайного выбора теория даёт $\rho/(1-\rho) = 9$, моделирование 8.9
- **Round robin** лучше случайного, потому что каждый сервер получает более регулярный поток, но длину очереди он не видит
- Знание длины очереди даёт ещё **в 4 раза** (4.8 → 1.1). Оговорка: точная мгновенная информация и один балансировщик

5.1 Power of two choices

- Выбираем **два** случайных узла и отправляем запрос менее загруженному. Mitzenmacher: две попытки вместо одной дают **экспоненциальное** улучшение, третья добавляет лишь константный множитель
- В нашей модели P2C снимает около **87%** разрыва между случайным выбором и least connections: $(8.9 - 2.1)/(8.9 - 1.1) \approx 0.87$
- Почему не всегда «самый короткий»: если информация устарела (несколько балансировщиков, метрики с задержкой), все идут в одну очередь. Такая **стадность** (herd behavior) описана в «How Useful Is Old Information?», а случайность её разрушает
- Где есть: nginx `random two` (по умолчанию метод `least_conn`, с версии 1.15.1), Envoy `LEAST_REQUEST` (выбирает из `choice_count` случайных хостов, по умолчанию 2)
- Горячий ключ P2C не лечит (лекция 3)

5.1 Веса и разогрев

- В nginx для весов 5:1:1 из каждых 7 запросов 5 уходят первому серверу. Со времён 2012 года работает **плавный** (smooth) вариант: последовательность `a a b a c a a`, а раньше было `c b a a a a a`
- Вес должен отражать **ёмкость** μ узла из нагрузочного теста (лекция 3), а не число ядер на глаз
- Новый или вернувшийся узел с пустыми кэшами не должен сразу получать полную долю. С least connections он вообще получит всё, пока у него 0 соединений
- **Slow start**: вес растёт от нуля до номинального за заданное время. nginx `slow_start` (в коммерческой версии), HAProxy `slowstart`, окно разогрева в Envoy

> Вес и разогрев это **статическая** подстройка. Всё, что зависит от текущей нагрузки, делает алгоритм выбора, а не веса.

5.1 Проверки состояния

	Пассивные	Активные
Как	по результатам реальных запросов: ошибки, таймауты	балансировщик сам периодически шлёт пробные запросы
Плюсы	нет лишнего трафика, видят реальные ошибки	замечают отказ до пользователей, возвращают узел в строй
Минусы	отказ обнаруживается ценой пользовательских ошибок	трафик проб; проба не равна реальному запросу
nginx	`max_fails` (по умолчанию 1) за `fail_timeout` (по умолчанию 10 с); узел считается недоступным на `fail_timeout`	периодические проверки (`health_check`) в коммерческой версии

- Время обнаружения при активных проверках: $\approx \mathrm{fall} \times \mathrm{interval}$ (+ таймаут пробы). Пример: интервал 2 с, 3 неудачи подряд, значит до 6 с. Пока узел не исключён, он получает долю $1/n$ запросов: при $n = 4$ и 1000 RPS это около 1500 ошибок
- Liveness (процесс жив, перезапустить) и readiness (готов принимать трафик) это **разные** проверки

5.1 Ловушки проверок и повторов

- **Глубокая проверка**, которая ходит в БД: БД замедлилась, проверки всех экземпляров провалились, балансировщик исключил **всех**, и приложение недоступно, хотя оно живо. Readiness должна проверять само приложение и критичные локальные ресурсы; состояние зависимостей лучше показывать метриками и деградацией
- **Паника**: Envoy при доле здоровых хостов ниже порога (по умолчанию 50%) перестаёт учитывать здоровье и раздаёт запросы всем, чтобы отказы не каскадировались
- **Повторы** на балансировщике: nginx по умолчанию передаёт запрос следующему серверу при ошибке связи или таймауте (`proxy_next_upstream error timeout`), но не повторяет `POST`, `LOCK`, `PATCH`, если запрос уже отправлен на сервер. Число попыток ограничивают `proxy_next_upstream_tries` и `proxy_next_upstream_timeout`
- Каждый повтор это дополнительная нагрузка именно тогда, когда система перегружена (лекция 2, подробнее в лекции 7)
- Если в группе один сервер, `max_fails` и `fail_timeout` в nginx игнорируются: единственный сервер недоступным не считается

5.1 Sticky sessions

Привязка клиента к узлу: по cookie (nginx `sticky cookie`, `route`, `learn`; в открытой версии с 1.29.6, раньше только в коммерческой), по хэшу адреса (`ip_hash`) или по ключу (consistent hash).

Цена	Что происходит
Перекос	один NAT или корпоративный прокси это много пользователей на одном узле. Если за ним 30% трафика и узлов 4, узел получает $0.3 + 0.7/4 = 47.5\%$, в 1.9 раза выше среднего (лекция 3, горячий ресурс)
Отказ и деплой	сессии узла теряются или переезжают без состояния; для плавного вывода нужен режим drain
Масштабирование	при scale-in узел нельзя убрать, пока к нему привязаны сессии; новые узлы получают нагрузку лишь с новыми клиентами
Состояние в экземпляре	возврат к stateful-слою из лекции 3

- Лучше **вынести состояние** во внешнее хранилище или подписанный токен. Sticky оправдан, когда потеря привязки это лишь промах кэша, и для долгоживущих соединений (WebSocket)

5.2 Зачем кэш: два числа

Пусть h это hit ratio, T_c и T_b задержки кэша и источника. В cache-aside промах платит **оба** обращения:

$$\bar{T} = T_c + (1 - h)\, T_b, \qquad \lambda_b = (1 - h)\, \lambda$$

Hit ratio h	Нагрузка на источник ($\lambda = 10\,000$ RPS)	Меньше, чем без кэша	Средняя задержка ($T_c = 1, T_b = 20$ мс)
нет кэша	10 000	1×	20 мс
0.5	5 000	2×	11 мс
0.9	1 000	10×	3 мс
0.99	100	100×	1.2 мс
0.999	10	1000×	1.02 мс

- От 90% к 99% задержка улучшилась с 3 до 1.2 мс, а нагрузка на источник упала **в 10 раз**: кэш прежде всего лечит **источник**

5.2 Кэш как часть ёмкости

- Если источник держит μ_b, а поток λ, то кэш **обязан** обеспечить

$$h \;\geq\; 1 - \frac{\mu_b}{\lambda}$$

Для $\lambda = 10\,000$ и $\mu_b = 2\,000$ нужно $h \geq 80\%$. Кэш перестаёт быть «ускорителем» и становится **обязательной частью ёмкости** (отсюда загадка)

- Хвост задержки задают промахи. Задержка смеси: $F(t) = h\,F_c(t) + (1-h)\,F_b(t)$. Если кэш много быстрее источника, **p-й процентиль** системы равен процентилю источника уровня $(p-h)/(1-h)$

h	0.90	0.95	0.98	0.99
p99 системы это процентиль источника	p90	p80	p50	граница: p99 уже у кэша

- Если SLO стоит на p99, а $h < 99\%$, то SLO определяется **задержкой источника**, а не кэша

5.2 Где размещается кэш



запрос идёт слева направо; чем правее кэш, тем он ближе к данным и тем дороже промах

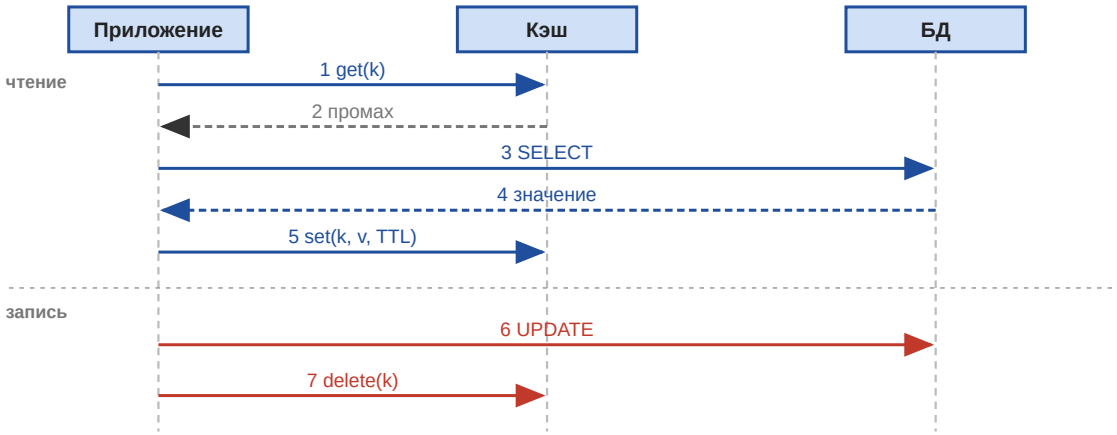
- На каждом уровне работает один и тот же приём, различаются **кто управляет** содержимым и **как** его инвалидировать: браузером управляют заголовки `Cache-Control` и `ETag` (RFC 9111), CDN и прокси это TTL и purge, приложением и Redis это код
- Сетевой переход к распределённому кэшу стоит порядка RTT внутри ЦОД (лекция 4), а локальный кэш в памяти процесса обходится без сети
- Нижний уровень есть всегда: буферный пул СУБД и страничный кэш ОС. Он уже кэширует горячие страницы, и это стоит измерить **до** добавления Redis

5.2 Локальный и распределённый кэш

	Локальный (в памяти процесса)	Распределённый (Redis, Memcached)
Доступ	вызов функции, без сети	сетевой переход
Общий для экземпляров	нет, у каждого свой	да
Объём	часть памяти приложения (конкуренция с кучей и GC)	отдельный пул, шардируется (лекция 3)
Согласованность	копии расходятся: окно устаревания у каждого узла своё	одна копия (с нюансами репликации)
Масштабирование	при n экземплярах каждый прогревает **свою** копию, hit ratio ниже	растёт вместе с кластером кэша
Отказ	теряется с процессом: деплой равен холодному кэшу	потеря узла это холодная доля ключей

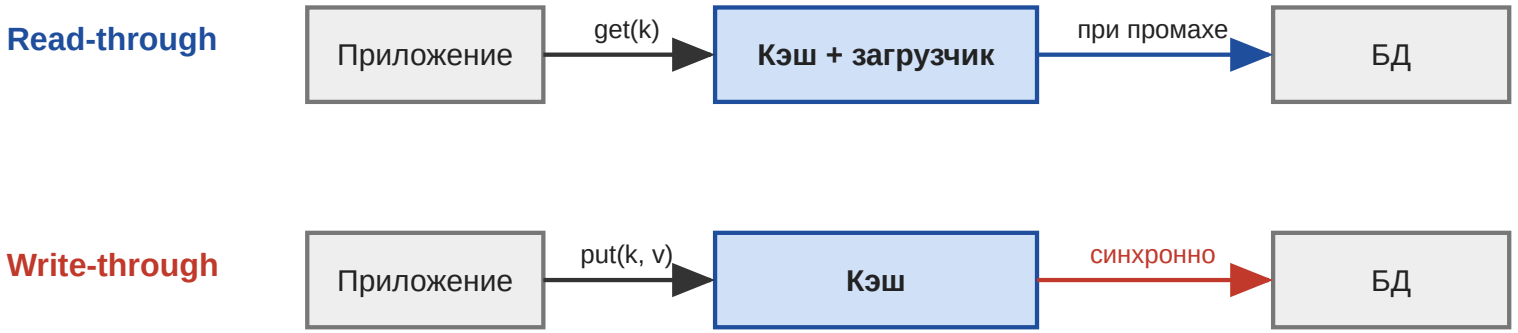
- Двухуровневая схема: короткоживущий локальный кэш (L1) перед распределённым (L2). Каждый уровень добавляет своё окно устаревания

5.2 Cache-aside



- Приложение **само** читает кэш и при промахе загружает данные из источника (lazy loading)
- Кэшируется только запрошенное; первый запрос всегда промах
- При записи: сначала БД, затем **delete** ключа, а не set. Facebook (NSDI 2013) выбрал удаление, потому что оно **идемпотентно**
- Кэш можно потерять без потери данных: источник остаётся истиной

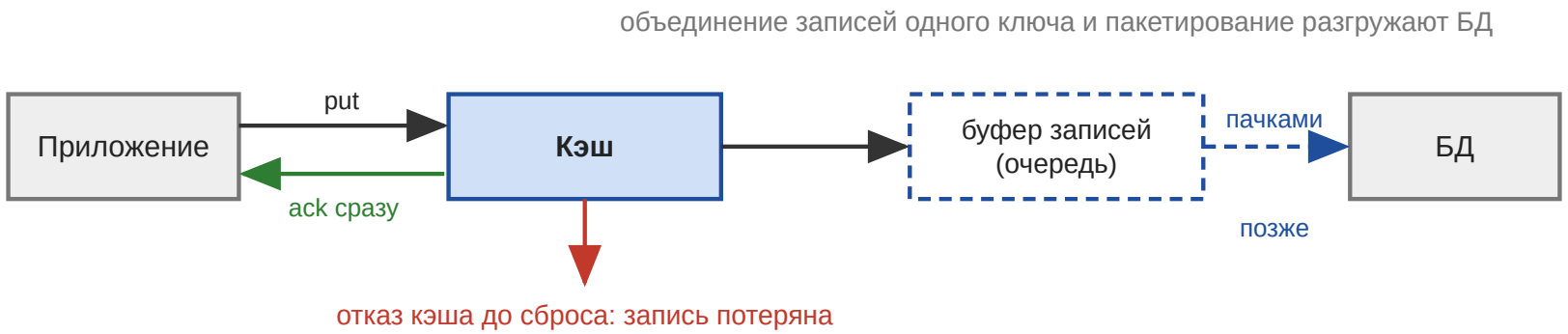
5.2 Read-through и write-through



ответ приложению — после записи и в кэш, и в БД: задержка записи = кэш + БД

- **Read-through**: приложение читает только кэш, а при промахе загружает **сам кэш** через загрузчик. Логика загрузки в одном месте и естественное место для объединения запросов (Caffeine: вычисление выполняется один раз даже при одновременных запросах)
- **Write-through**: запись идёт через кэш, который синхронно пишет в источник. Кэш не отстаёт от источника, но задержка записи равна кэш + БД
- Риск: запись в БД прошла, а в кэш нет (или наоборот): кэш и источник расходятся

5.2 Write-behind и write-around



- **Write-behind**: запись подтверждается после кэша, в источник уходит позже, пачками. Плюсы: быстрая запись, сглаживание пиков, повторные записи одного ключа сливаются в одну
- Минусы: при отказе кэша **теряется** всё, что ещё не сброшено; порядок записей и обработка ошибок записи в БД (кому сообщить?). Подходит для счётчиков и телеметрии, не для платежей
- **Write-around**: запись идёт только в БД, кэш не трогается; подходит для данных, которые пишутся и редко читаются

5.2 Шаблоны: сравнение

Шаблон	Кто загружает кэш	Путь записи	Задержка записи	Главный риск	Типичное применение
Cache-aside	приложение	БД, затем delete	как у БД	устаревание, гонка	профили, справочники
Read-through	кэш (загрузчик)	как у cache-aside	как у БД	зависит от библиотеки	то же, плюс объединение запросов
Write-through	—	кэш и БД синхронно	кэш + БД	дорогая запись, частичный отказ	чтение сразу после записи
Write-behind	—	кэш, БД позже	как у кэша	**потеря данных**, порядок	счётчики, телеметрия
Write-around	—	сразу БД	как у БД	промах после записи	редко читаемое

Шаблон выбирается по **цене ошибки**: допустима ли потеря, устаревание, лишняя задержка записи.

5.2 TTL и окно устаревания

- TTL это **верхняя граница** устаревания, если нет другой инвалидации. Цена: ключ раз в TTL промахивается
- Для ключа с потоком r запросов/с (пуассоновским) и фиксированным TTL T (отсчёт от загрузки):

$$h_{\text{key}} = \frac{rT}{1 + rT}$$

Поток ключа r	$T = 60$ с	Комментарий
1 запрос/с	98.4%	горячий ключ
0.1 запрос/с	85.7%	
0.01 запрос/с	**37.5%**	длинный хвост почти не попадает в кэш

- Локальные кэши: при $n = 10$ экземплярах за round robin каждый видит r/n. Ключ с $r = 1$ даёт уже $0.1 \cdot 60/(1 + 0.1 \cdot 60) = 85.7\%$ вместо 98.4%

5.2 Инвалидация

Способ	Как	Цена
Только TTL	ждём истечения	окно устаревания равно TTL, нет координации
Удаление при записи	запись в БД, затем `delete(k)`	потерянное удаление держит устаревшее до TTL; TTL остаётся страховкой
Обновление при записи	пишем новое значение в кэш	гонки порядка записей; Facebook выбрал удаление
Версионированный ключ	ключ содержит версию: `user:42:v17`	не нужно удалять, но нужно знать актуальную версию
События (CDC, pub/sub)	журнал изменений БД удаляет ключи	асинхронность, отставание, ещё один компонент
Отдать устаревшее	`stale-while-revalidate`, `stale-if-error` (RFC 5861)	осознанное устаревание ради скорости и доступности

- Кэш это **реплика с окном устаревания** (лекция 4): инвалидация это вопрос согласованности

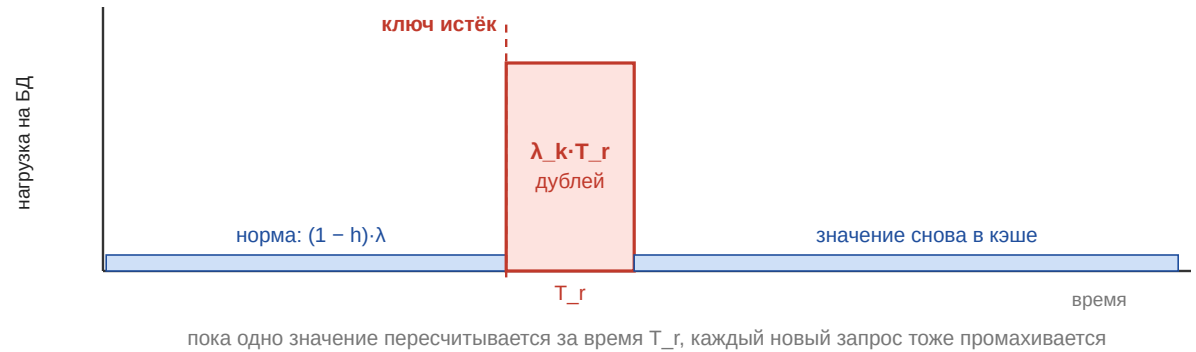
5.2 Гонка: устаревшая запись в кэш



кэш хранит v1, БД хранит v2: расхождение до истечения TTL

- Ключ «залип» с устаревшим значением. Facebook решил это **лизами**: при промахе читатель получает токен, `delete` его аннулирует, а `set` по аннулированному токену отклоняется. Те же лизы выдаются не чаще раза в 10 с на ключ, остальные ждут (защита от thundering herd); по данным статьи, пик запросов к БД упал с 17 тыс/с до 1.3 тыс/с
- Другие приёмы: короткий TTL как страховка, запись с проверкой версии

5.2 Cache stampede



пока одно значение пересчитывается за время T_r, каждый новый запрос тоже промахивается

- Популярный ключ истёк, одновременные запросы **все** промахиваются и все идут в источник за одним значением (cache stampede, dogpile effect)
- Оценка по закону Литтла (лекция 2): пока значение пересчитывается T_r, приходит $\lambda_k T_r$ запросов. Для $\lambda_k = 10\,000$ RPS и $T_r = 200$ мс это **2 000** дублей вместо одного запроса
- Под нагрузкой пересчёт замедляется, T_r растёт, дублей становится больше: **положительная обратная связь** (лекции 2 и 7)

5.2 Защита от stampede

Приём	Идея	Цена
Single-flight, блокировка пересчёта	один пересчитывает, остальные ждут результат (nginx `proxy_cache_lock`, Go `singleflight`)	ожидание; в nginx по таймауту (5 с) запрос уходит к источнику, но ответ не кэшируется
Отдать устаревшее на время обновления	nginx `proxy_cache_use_stale updating` и `proxy_cache_background_update`, RFC 5861	устаревание на время обновления
Раннее вероятностное обновление (XFetch)	обновлять, если $t - \delta\beta \ln u \geq t_{\exp}$, где $u \sim U(0,1)$, δ время пересчёта, $\beta = 1$	нужно хранить δ; гарантия вероятностная
Джиттер TTL	$T(1 \pm \varepsilon)$: ключи не истекают разом	один горячий ключ не защищён
Лизы	как у Facebook	нужна поддержка в кэше

- Комбинация: single-flight для холодного промаха, stale-while-revalidate или XFetch для тёплых ключей

5.2 Термины: не путать

Термин	Что происходит	Что помогает
Thundering herd	много ожидающих разом пробуждены одним событием, а обработать может один (исторически термин ОС)	один «будильник»: блокировка, очередь
Cache stampede, dogpile	один горячий ключ истёк, дубли пересчёта	single-flight, stale-while-revalidate, XFetch
Cache avalanche	много ключей истекают одновременно или кэш целиком недоступен	джиттер TTL, прогрев, реплики кэша, ограничение нагрузки на источник (лекция 7)
Cache penetration	запросы к ключам, которых нет ни в кэше, ни в БД	кэш отрицательных ответов с коротким TTL, фильтр Блума, валидация ключей

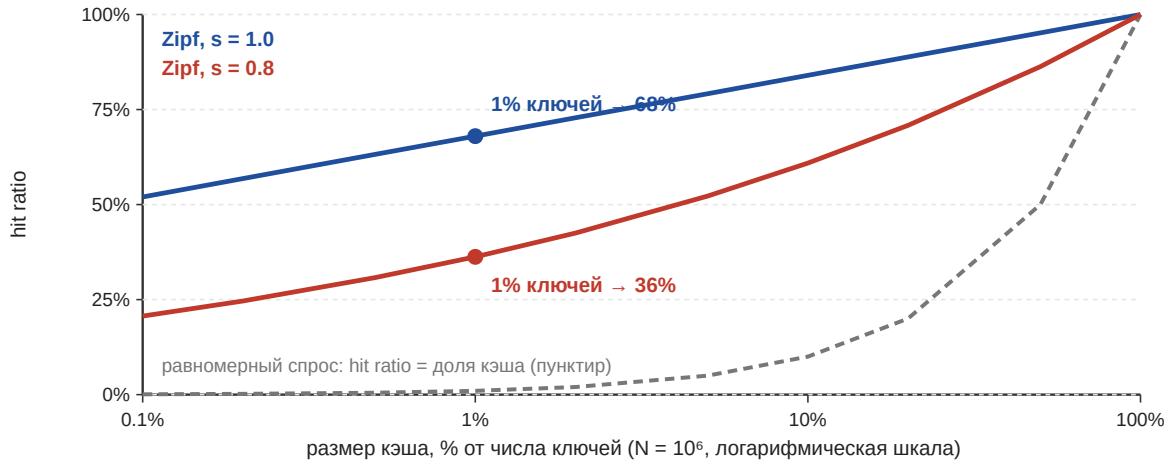
- Названия **не стандартизованы**, в литературе встречаются и другие. Важнее описывать механизм: что истекло, сколько ключей, есть ли они в источнике

5.2 Разгадка

- В норме БД получала $(1 - 0.95) \cdot 10\,000 = 500$ RPS. С **пустым** кэшем $h = 0$, и БД получила **10 000** RPS: в 20 раз больше нормы и в 5 раз больше её предела (2 000)
- БД замедлилась, таймауты, повторы клиентов добавили нагрузки. Даже когда кэш наполнился, очередь и повторы держали БД на пределе: перегрузка **поддерживает сама себя** (обратная связь ретраев, лекция 2; в литературе это метастабильный сбой)
- Реальный случай: OpenAI (январь 2026) описывает инциденты PostgreSQL по одной схеме: массовые промахи кэша после отказа слоя кэширования, затем рост задержек, таймауты и повторы, усиливающие нагрузку
- Защита: **лизы и блокировки** (в OpenAI только один читатель с промахом идёт в PostgreSQL, остальные ждут), ограничение частоты запросов, прогрев кэша перед включением трафика, постепенный возврат трафика, реплики кэша

> Кэш с $h \geq 1 - \mu_b/\lambda$ это **зависимость**: его отказ равен перегрузке источника. Считайте источник **без** кэша.

5.2 Что кэшировать



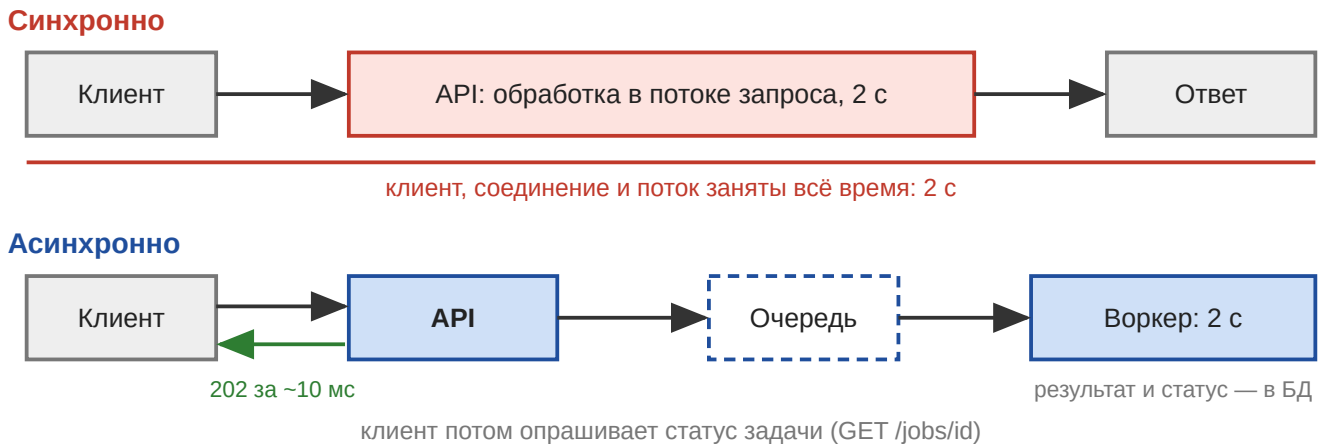
- Модель: идеальный кэш хранит самые популярные ключи; $N = 10^6$. При $s = 1$ кэш из **1%** ключей даёт 68% попаданий, при $s = 0.8$ только 36%. Реальные потоки это Zipf-подобные распределения с показателем, который **зависит от потока** (Breslau et al., 1999)
- Реальные политики вытеснения хуже идеала. Redis по умолчанию не вытесняет (`noeviction`), а LRU приближает выборкой из `maxmemory-samples` = 5 ключей
- Кэш не поможет при равномерном спросе, огромном рабочем наборе, частых записях и персонализированных ответах

5.2 Мостики

Где ещё этот приём	Что кэшируется	Как устроена инвалидация
Процессор (лекция 10)	строки памяти в L1–L3	протокол когерентности, write-back
ОС и СУБД (лекция 6)	страницы диска: page cache, буферный пул	вытеснение, журнал
DNS	соответствия имён и адресов	TTL записи (окно устаревания задано явно)
CDN и браузер	ответы HTTP	`Cache-Control`, `ETag`, purge
Приложение (эта лекция)	результаты запросов и вычислений	TTL, удаление, версии

Везде один компромисс: **скорость и разгрузка** против **свежести и сложности**. Меняются масштаб и цена ошибки.

5.3 Синхронный и асинхронный путь



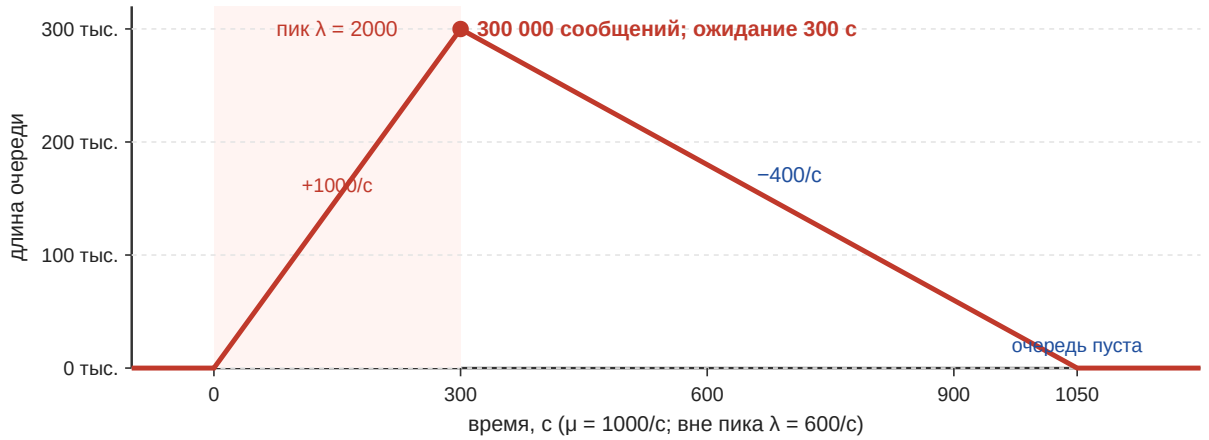
- Синхронно клиент ждёт всю обработку, и всё это время заняты соединение, поток и память. По закону Литтла (лекция 2) в полёте $L = \lambda W = 500 \cdot 2 = 1000$ запросов
- Асинхронно API принимает работу, кладёт в очередь и отвечает за ~10 мс: на API в полёте $500 \cdot 0.01 = 5$ запросов
- **Работа не исчезла**: воркерам всё равно нужно выполнить 500×2 с работы в секунду. Асинхронность меняет, **где и когда** мы ждём, а не количество работы

5.3 Что даёт асинхронность и что нет

Даёт	Не даёт, или стоит
Ответ клиенту не зависит от времени обработки	Результата нет сразу: нужен статус, опрос или уведомление
Очередь сглаживает пики (лекция 2)	**Не добавляет ёмкости**: при $\lambda_{\text{ср}} > \mu$ очередь растёт без предела
Развязка отказов: недоступный обработчик не роняет API	Новые отказы: дубли, потери, перестановки, «отравленные» сообщения
Независимое масштабирование API и воркеров	Больше компонентов: брокер, мониторинг, эксплуатация
Приоритеты и пакетирование	Задержка выше: время в очереди плюс обслуживание

> Очередь сглаживает **дисперсию** потока, но не **дрейф**: если средняя нагрузка выше ёмкости, очередь растёт, пока не кончится память.

5.3 Сглаживание пика: числа



- $\mu = 1000$ задач/с, обычно $\lambda = 600$, 5 минут пик $\lambda = 2000$. Очередь растёт со скоростью $\lambda - \mu = 1000$/с и через 300 с достигает **300 000**; последнее сообщение пика ждёт $L/\mu = 300$ с
- После пика очередь убывает со скоростью $\mu - \lambda = 400$/с и опустеет через $300\,000/400 = 750$ с (12.5 мин)
- Это допустимо, только если пользователь согласен ждать: у асинхронной работы нужен свой **SLO по времени до завершения**

5.3 Очередь прячет перегрузку

- Длина очереди это **запаздывающий** индикатор. Следите за **возрастом старейшего сообщения**: по Литтлу ожидание $W \approx L/\mu$
- Ограничьте очередь и **время жизни** сообщения: задача, которую уже не ждут, это пустая работа (goodput, лекция 2)
- Решите заранее, что происходит при переполнении: отказ (429 или 503) или **обратное давление** на производителя (лекция 7). Молчаливый рост очереди это не вариант
- Метрики: скорость поступления, скорость обработки, глубина, возраст старейшего, число повторов, размер очереди «мёртвых писем»

> «Очередь не растёт» ещё не значит «система здорова»: очередь может быть пуста, потому что клиенты уже отвалились по таймауту.

5.3 API асинхронной задачи

```
POST /v1/jobs        →  202 Accepted       Location: /v1/jobs/7f3a
GET  /v1/jobs/7f3a   →  200 OK             {"status": "running"}   Retry-After: 5
GET  /v1/jobs/7f3a   →  303 See Other      Location: /v1/results/7f3a
```

- `202 Accepted` значит, что запрос принят, но обработка не завершена и может не завершиться успешно (RFC 9110, §15.3.3). `Location` указывает на ресурс статуса, `Retry-After` подсказывает частоту опроса (Azure Architecture Center, Asynchronous Request-Reply)
- Альтернативы опросу: webhook, SSE, WebSocket
- Состояние задачи (`queued → running → done / failed`) хранится в **БД**, а не в памяти воркера: тогда его переживают рестарты (так устроен и сквозной проект: клиент получает идентификатор и узнаёт состояние)

5.3 Надёжность асинхронного пути

Проблема	Как проявляется	Приём
Доставка «хотя бы раз»	воркер упал после обработки, до подтверждения: сообщение придёт снова	**идемпотентный** обработчик: ключ идемпотентности и таблица обработанных
Двойная запись (БД и брокер)	запись в БД прошла, публикация нет, или наоборот	**transactional outbox**: событие пишется в БД в той же транзакции, отдельный процесс публикует
«Отравленное» сообщение	всегда падает и блокирует очередь	лимит повторов и очередь «мёртвых писем» (лекция 6)
Порядок	параллельные воркеры переставляют сообщения	один ключ идёт в одну секцию и к одному обработчику (раздел 5.4)
Потерянный статус	клиент не знает исхода	состояние задачи в БД

- Идемпотентность на практике (Stripe): сервер сохраняет код и тело **первого** ответа для ключа, повтор с тем же ключом возвращает тот же результат
- Практика: доставка «хотя бы раз» плюс идемпотентный обработчик (лекция 4: таймаут не говорит, выполнилась ли операция)

5.3 Пакетирование: модель

Фиксированная цена операции a (fsync, RTT, коммит, системный вызов) и цена элемента c. Пачка из b элементов:

$$t(b) = a + b\,c, \qquad X(b) = \frac{b}{a + b\,c} \xrightarrow{b \to \infty} \frac{1}{c}$$

Размер пачки b	1	10	100	1000
Время пачки, мс ($a = 1, c = 0.02$)	1.02	1.2	3.0	21
Пропускная способность, элементов/с	980	8 333	33 333	47 619

- Выигрыш **насыщается**: от 100 до 1000 пропускная способность выросла лишь в 1.4 раза, а время пачки в 7 раз
- Цена: первый элемент ждёт, пока пачка наполнится, плюс $t(b)$. Это ещё одна очередь, и она добавляет к хвосту задержки
- Примеры: многострочный `INSERT` и `COPY`, пачка сообщений, алгоритм Нейгла в TCP (RFC 896)

5.3 Пачка: по размеру или по времени

- Пачка закрывается по **размеру** B или по **времени** T, что наступит раньше. При потоке λ она наполняется за B/λ
- Пример: $\lambda = 2000$ сообщений/с, $B = 100$, $T = 20$ мс. Наполнение заняло бы $B/\lambda = 50$ мс, больше T: пачка закрывается **по времени**, средний размер $\lambda T = 40$. Увеличивать B выше 40 бессмысленно
- Kafka-продюсер: `batch.size` (в байтах, по умолчанию 16 384) и `linger.ms` (сколько ждать заполнения пачки; до Kafka 4.0 по умолчанию 0, с версии 4.0 это 5 мс)
- Задержка T добавляется **каждому** сообщению в худшем случае. Подбирайте T из бюджета задержки (лекция 1: SLO), а не «на глаз»

> Пакетирование **обменивает задержку на пропускную способность**. Это выгодно, пока пропускная способность и есть ограничение.

5.3 Объединение одинаковых запросов



- k одновременных одинаковых запросов дают **одно** обращение к источнику, результат получают все. Нагрузка на источник: $k \to 1$
- Где: в приложении (Go `singleflight.Group.Do(key, fn)`, `cache.get(key, loader)` в Caffeine), на прокси (nginx `proxy_cache_lock`)
- Ограничения: **общая судьба** (ошибка или таймаут лидера достаётся всем ожидающим); ключ должен однозначно определять результат; это не кэш: после завершения следующий запрос снова идёт к источнику

5.3 Три приёма: чем отличаются

	Кэширование	Пакетирование	Объединение запросов
Что делает	хранит результат на будущее	объединяет **разные** операции в одну	объединяет **одинаковые** одновременные запросы
Выигрыш	повторные запросы не доходят до источника	амортизация фиксированной цены операции	дубли в момент промаха не доходят до источника
Цена	устаревание, память	ожидание заполнения пачки	общая судьба, ожидание лидера
Работает, когда	запросы повторяются во времени	есть поток операций	дубли приходят одновременно

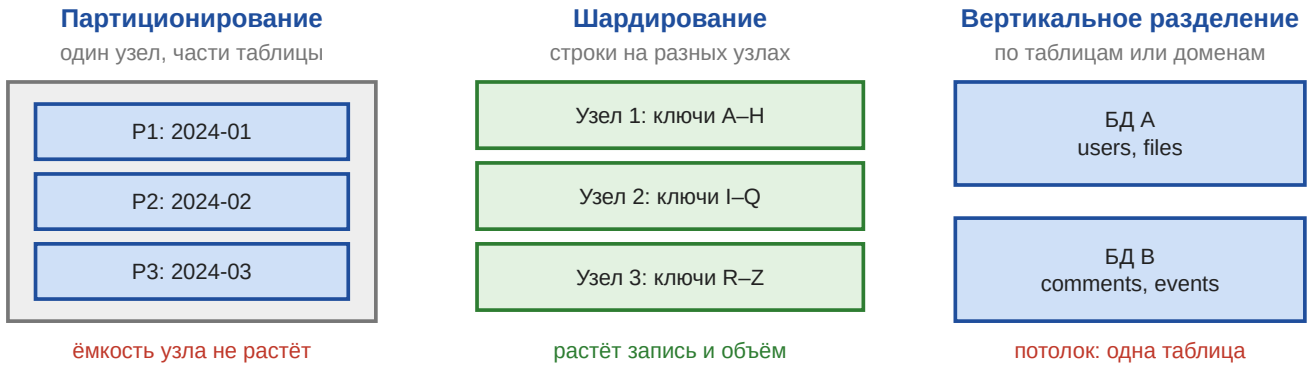
- Комбинации: кэш + single-flight лечат stampede; write-behind это кэш + пакетирование

5.3 Мостики

Область	Тот же приём
Сеть и ядро ОС	объединение прерываний сетевой картой, алгоритм Нейгла в TCP: склеивание мелких отправок ценой задержки
Журнал СУБД (лекция 6)	несколько транзакций делят один сброс журнала на диск (group commit)
Кэши процессора (лекция 10)	буфер записи и write-back: запись откладывается и сливается
MapReduce (лекция 8)	combiner: агрегация перед передачей по сети
Брокеры (лекция 6)	пачки сообщений, подтверждения пачками

Везде фиксированная цена операции делится на b элементов, а платит за это **ожидание**.

5.4 Термины: разбиение данных



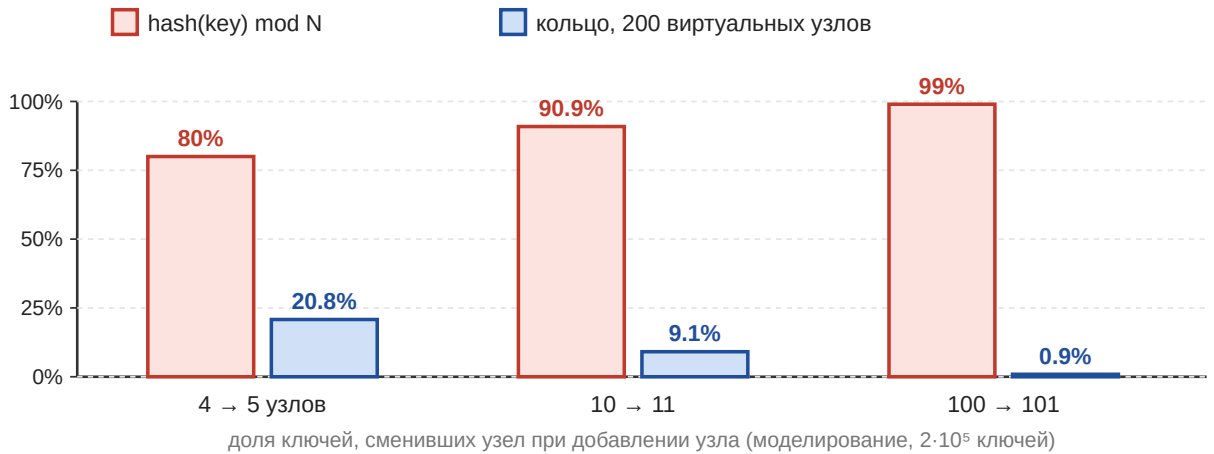
- **Партиционирование**: части таблицы внутри одной СУБД (PostgreSQL: по диапазону, списку или хэшу). Выигрыш: отсечение секций, дешёвое удаление старых, меньшие индексы. **Ёмкость узла не растёт**
- **Шардирование**: горизонтальные секции на **разных** узлах; растут объём и пропускная способность записи (лекция 4)
- **Вертикальное разделение**: по таблицам или доменам в разные БД (Figma: к концу 2022 года около десятка БД). Потолок: одна таблица
- В лекции 4 слово «секционирование» покрывало оба смысла; дальше шардирование это секции на разных узлах

5.4 Выбор ключа секционирования

Схема	Как	Плюсы	Минусы
По диапазону	ключи делятся на отрезки	диапазонный запрос читает мало секций	монотонные ключи (время, автоинкремент): вся запись в последнюю секцию
По хэшу	$h(\text{key}) \bmod N$	равномерно	диапазонный запрос идёт во **все** секции; смена N перемешивает ключи
Справочник	таблица «ключ → секция»	любое размещение, ручная миграция	справочник сам узкое место и точка отказа
Хэш-слоты	ключ → слот, слоты → узлы	миграция слотами; Redis Cluster: $\text{CRC16}(\text{key}) \bmod 16384$	число слотов фиксировано

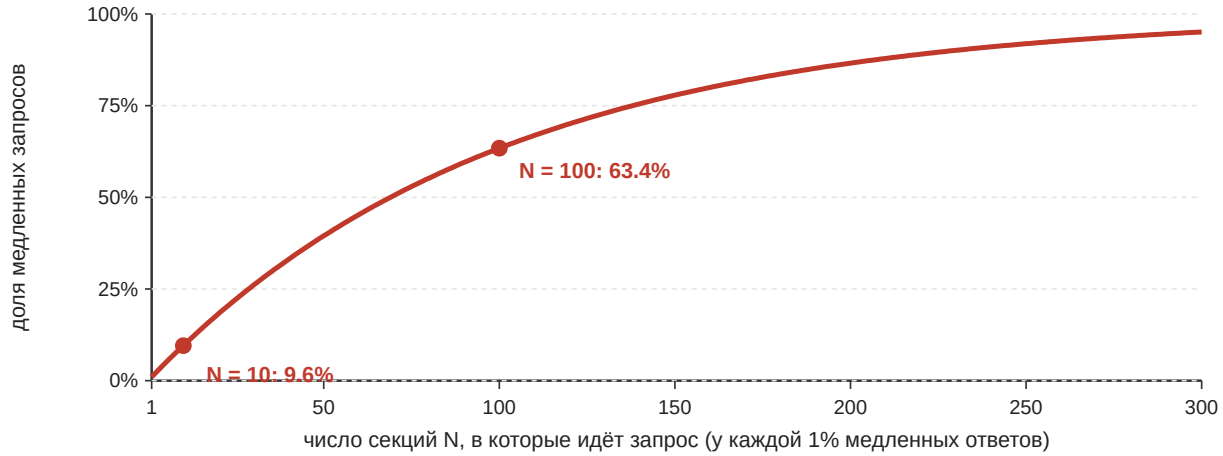
- Выбор ключа это выбор **дешёвых запросов**: запрос без ключа секционирования идёт во все секции

5.4 Перебалансировка



- $h(\text{key}) \bmod N \to N+1$: на месте остаётся доля $1/(N+1)$, **перемещается** $N/(N+1)$ ключей. Для кэш-кластера это почти полный холодный старт
- Консистентное хэширование (Karger et al., 1997): перемещается лишь доля порядка $1/(N+1)$. Виртуальные узлы выравнивают нагрузку: при 10 узлах максимум от среднего 4.0× (1 точка на узел), 1.26× (10 точек), 1.15× (200 точек)
- Альтернатива: **много фиксированных секций** (Redis: 16 384 слота), перемещаем целые секции. nginx: `hash $key consistent` (ketama)

5.4 Цена: запрос во все секции



- Запрос без ключа идёт во все N секций (scatter-gather), задержка равна **максимуму**. Если у каждой секции доля медленных ответов $p = 1\%$, медленными будут $1 - (1 - p)^N$ запросов: $N = 10$ даёт 9.6%, $N = 100$ даёт **63.4%** (Dean, Barroso, 2013)
- Лекарства: запросы с ключом секционирования, параллельные обращения с таймаутом и частичным ответом, hedged-запросы (лекция 3)

5.4 Цена: join, транзакции, индексы

Операция	Что меняется при шардировании
JOIN между секциями	данные идут по сети; связанные таблицы размещают по **одному ключу** (Figma: группы таблиц «colos»)
Транзакция на нескольких секциях	распределённая фиксация (двухфазная) дорога и снижает доступность (лекция 4) либо согласованность в итоге
Вторичный индекс	локальный: запрос идёт во все секции; глобальный: запись затрагивает несколько секций (Kleppmann, гл. 6)
Уникальность не по ключу	нужна глобальная проверка
Миграции схемы, бэкапы, мониторинг	операций становится $\times N$

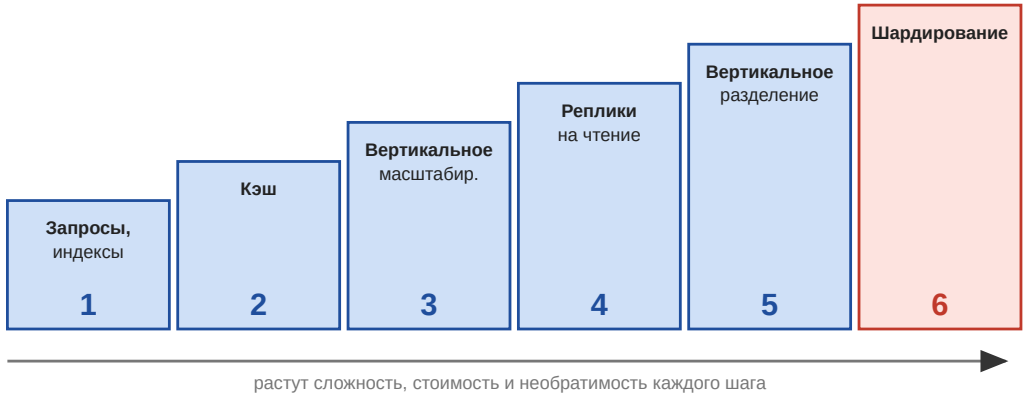
Всё это **не** проявляется в бенчмарке на одной секции, поэтому цену шардирования надо считать до его начала.

5.4 Горячие ключи и разделы

Причина	Приём	Цена
Часто читаемый ключ	кэш перед секцией; реплики секции	устаревание (5.2)
Часто **записываемый** ключ (счётчик)	разбить ключ на k подключей `key#0…k-1`, при чтении суммировать; write-behind и пакетирование (5.3)	чтение дороже в k раз
Монотонный ключ (время)	составной ключ (хэш-префикс, время)	диапазонные запросы идут в несколько секций
Вирусный всплеск	ограничение частоты и изоляция (лекция 7), кэш	часть запросов отклоняется

- Метрики **по секциям**: средняя нагрузка скрывает перекос (лекция 3). Горячий раздел исправляется ключом и кэшем, а не добавлением узлов

5.4 Лестница масштабирования данных



- **Figma**: в 2020 году одна БД PostgreSQL на самом крупном инстансе AWS; к концу 2022 года кэш, реплики для чтения и около десятка вертикально разделённых БД; затем горизонтальное шардирование (первая таблица заняла девять месяцев)
- **OpenAI** (январь 2026): один primary PostgreSQL и около 50 реплик на 800 млн пользователей; шардируемые write-heavy нагрузки вынесены в шардируемые системы
- Порядок типичный, а не закон: каждая ступень дешевле и обратимее следующей

5.4 Когда шардирование преждевременно

Сначала проверьте (дешевле и обратимо)	Признаки, что пора
планы запросов и индексы, N+1 запросов	запись упирается в один primary **после** оптимизаций (реплики пишут то же самое: лекция 4)
кэш (5.2) и реплики для чтения	рабочий набор не помещается в память узла, а промахи дороги
пул соединений (OpenAI: PgBouncer, среднее время подключения 50 мс → 5 мс)	время восстановления: 10 ТБ при 500 МБ/с это около 5.6 ч, и это больше допустимого RTO
вертикальное масштабирование, архивирование старых данных	радиус отказа: падение единственной БД роняет всё
вынести тяжёлую запись в очередь (5.3)	требования к размещению данных по регионам

- Предел PostgreSQL: отношение (таблица) до 32 ТБ при блоке 8 КБ, размер базы не ограничен (документация). Практические пределы наступают раньше: бэкапы, autovacuum, время восстановления

5.4 Мостики

Где ещё	Идея
Kafka (лекция 6)	ключ определяет секцию; порядок гарантируется только внутри секции
MapReduce (лекция 8)	shuffle: данные разбиваются по ключу между редьюсерами
Dynamo (лекция 4)	консистентное хэширование и репликация на соседние узлы кольца
USL (лекция 3)	секционирование убирает общий ресурс, который даёт α
Кэши процессора и NUMA (лекция 10)	данные закрепляются за ядром, чтобы не делить их

Везде секционирование **уменьшает область общего состояния**. Цена: операции, которым нужен весь набор данных, становятся дорогими.

5.5 Цена каждого компонента

Компонент	Что решает	Что добавляет
Балансировщик	распределение нагрузки, отказ экземпляров	сетевой переход, точку отказа, настройку проверок, перекос при sticky
Кэш	нагрузку на источник и задержку	устаревание, холодный старт, stampede, память, **скрытую зависимость**
Очередь	пики, развязку, тяжёлую работу	ожидание, дубли, статус, мониторинг, эксплуатацию
Шардирование	предел одной БД по записи и объёму	запросы между секциями, миграции, горячие секции, расходы $\times N$
Микросервис	независимые релизы и масштабирование	сеть, частичные отказы, версионирование, наблюдаемость

> Компонент это **кредит**: он решает измеренную проблему сегодня и берёт проценты сложностью завтра. Брать его стоит под **конкретное измеренное ограничение**.

5.5 Преждевременное шардирование

- **Симптомы**: шардируем «на вырост» до измерений; ключ выбран без анализа запросов; нагрузка на одну БД далека от её ёмкости
- **Цена ошибки**: смена ключа это перенос **всех** данных. Figma: девять месяцев на первую таблицу при готовой команде и уже вертикально разделённой базе. OpenAI: шардирование существующих нагрузок потребовало бы изменить сотни endpoint-ов и заняло бы месяцы или годы, поэтому PostgreSQL остался с одним primary, а новые write-heavy нагрузки уходят в шардируемые системы
- **Что делать**: подняться по лестнице (5.4) и шардировать, когда **измерение** показывает исчерпание запаса
- «Преждевременно» не значит «никогда»: Figma шардировала, когда вертикального разделения стало мало. Но ключ нужно **проектировать заранее**, чтобы потом не переделывать

> Шардирование трудно отменить, и ошибка в ключе дорого стоит. Это решение с высокой ценой отката.

5.5 Микросервисы без необходимости

- **Доступность** цепочки это произведение: $A = \prod A_i$. Десять последовательных сервисов по 99.9% дают **99.0%**: около 87 ч простоя в год вместо 8.8 ч
- **Задержка и хвост**: каждый вызов это сетевой переход (лекция 4); при веере из 100 вызовов с $p = 1\%$ медленных ответов медленны 63% запросов (5.4)
- Fowler (2015): *MicroservicePremium* это цена управления набором сервисов; не рассматривайте микросервисы, пока система не слишком сложна для монолита. *MonolithFirst*: почти все успешные микросервисные истории начинались с монолита, который вырос и был разделён
- **Prime Video** (2023): сервис мониторинга качества потоков на Step Functions и Lambda упёрся в предел примерно при 5% ожидаемой нагрузки; перенос в один процесс снизил стоимость инфраструктуры более чем на 90%. Это **один** сервис, а не отказ компании от микросервисов
- Типичные признаки «распределённого монолита»: цепочки синхронных вызовов, общая БД, совместные релизы

5.5 Инфраструктура без измеренной проблемы

Добавили	Вопрос, на который нет ответа без измерения
Кэш	какой hit ratio, какую долю нагрузки он снимает, повторяются ли ключи (5.2)?
Очередь, Kafka	какой пик, какая задержка допустима, сколько работы можно отложить?
Реплики, шардирование	упирается ли чтение или запись, какой ресурс насыщен?
Автомасштабирование	узкое место в приложении или ниже по цепочке (лекция 3)?
Микросервисы	какую проблему решаем: ёмкость, независимые релизы, изоляцию отказов?

- Цикл: нагрузка, измерение, узкое место, **наименее дорогое** изменение, повторное измерение. Изменение готово, когда есть метрика «до и после» и условие отката
- Кнут (1974): «premature optimization is the root of all evil», но сразу же: не упускать критические 3%. Их находят **измерением**

5.5 Другие типичные ловушки

Ловушка	Почему плохо	Что вместо
Кэш как источник истины	при вытеснении и перезапуске данные теряются	истина в хранилище, кэш восстановим
Неограниченная очередь	память до OOM, ожидание без предела (лекция 2)	ограниченная очередь и политика переполнения
Повторы на каждом уровне	3 уровня по 3 попытки дают до $3^3 = 27$ обращений к нижнему уровню на один запрос; OpenAI описывает такие повторы как усилитель нагрузки	бюджет повторов, backoff с jitter (лекция 7)
Глубокая проверка состояния	каскадное исключение всех узлов (5.1)	проверка только самого приложения
Sticky вместо внешнего состояния	перекос и потеря сессий (5.1)	состояние во внешнем хранилище
Общая БД между сервисами	связь через схему; общий ресурс даёт α в USL (лекция 3)	у каждого сервиса свои данные
Кэш без плана холодного старта	загадка этой лекции	считать источник без кэша (5.2)

5.5 Карта выбора: ограничение → механизм

Измеренное ограничение	Механизм	Цена	Раздел
Экземпляр приложения насыщен по CPU, состояние вынесено	больше экземпляров и балансировщик	хоп, проверки, узкое место ниже	5.1, лекция 3
Узлы загружены неравномерно при разной длине запросов	least connections или P2C вместо round robin	счётчики, устаревшая информация	5.1
Чтения повторяются, источник насыщен чтением	кэш (cache-aside), реплики	устаревание, stampede, холодный старт	5.2
Одинаковые запросы приходят одновременно	single-flight, объединение	общая судьба	5.2, 5.3
Пики коротки, работу можно отложить	очередь и воркеры	ожидание, идемпотентность, статус	5.3
Велика фиксированная часть цены операции	пакетирование	ожидание заполнения	5.3
Запись упирается в один primary после оптимизаций	вертикальное разделение, затем шардирование	запросы между секциями, ключ	5.4
Один ключ перегружает секцию	кэш перед секцией, разбиение ключа	устаревание, чтение дороже в k раз	5.4

Нет измеренного ограничения: нет оснований для механизма.

Чек-лист инженера

1. Назовите **измеренное ограничение** (какой ресурс насыщен) до выбора механизма
2. Балансировка: least connections или P2C для запросов разной длины; **slow start** для новых узлов; проверки без зависимостей; повторять только идемпотентные запросы
3. Кэш: посчитайте $h \geq 1 - \mu_b/\lambda$ и ответьте, что будет с источником при **пустом** кэше
4. Для каждого кэшируемого объекта задайте **окно устаревания** и способ инвалидации; для горячих ключей: single-flight, jitter, stale-while-revalidate
5. Асинхронность и пакетирование: ограниченная очередь, **возраст старейшего сообщения** в метриках, идемпотентный обработчик, статус задачи в БД; пачка по размеру **и** по времени
6. Шардируйте **последним**: сначала запросы, кэш, реплики, вертикальное разделение; ключ выбирайте по запросам; метрики ведите по секциям
7. Записывайте у каждого нового компонента: что добавляет, как измеряем, как откатываем

Заключение

Тема	Главное
Балансировка	L4 раздаёт соединения, L7 запросы; в модели least connections и P2C дают ожидание 1.1 и 2.1 против 4.8 и 8.9 у round robin и случайного выбора; проверки состояния без зависимостей
Sticky sessions	цена: перекос, потеря сессий, блокировка scale-in; лучше вынести состояние
Кэш	$\lambda_b = (1-h)\lambda$; p99 определяется кэшем только при $h > 99\%$; кэш это часть ёмкости: $h \geq 1 - \mu_b/\lambda$
Устаревание и stampede	TTL ограничивает окно; $h_{\text{key}} = rT/(1+rT)$; `delete` вместо `set`; $\lambda_k T_r$ дублей; single-flight, XFetch, jitter
Асинхронность	не добавляет ёмкости; очередь это буфер плюс ожидание; идемпотентность, outbox, статус
Пакетирование, объединение	$X(b) = b/(a+bc)$, пачка по размеру или времени; $k \to 1$ запросов к источнику
Разделение данных	ключ выбирает дешёвые запросы; $\bmod N$ перемещает $N/(N+1)$ ключей; $1-(1-p)^N$
Анти-паттерны	решение до измерения; каждый компонент это кредит

Список источников (1/2)

- Mitzenmacher. *The Power of Two Choices in Randomized Load Balancing* (IEEE TPDS 12(10), 2001)
- Mitzenmacher. *How Useful Is Old Information?* (IEEE TPDS 11(1), 2000)
- Eisenbud et al. *Maglev: A Fast and Reliable Software Network Load Balancer* (NSDI 2016)
- Karger et al. *Consistent Hashing and Random Trees* (STOC 1997)
- nginx Documentation. *ngx_http_upstream_module*, *ngx_http_proxy_module* (`random two`, `sticky`, `max_fails`, `proxy_cache_lock`, `proxy_next_upstream`); коммит *Upstream: smooth weighted round-robin balancing* (2012)
- Envoy Documentation. *Load balancing* (least request, panic threshold)
- Nishtala et al. *Scaling Memcache at Facebook* (NSDI 2013)
- Vattani, Chierichetti, Lowenstein. *Optimal Probabilistic Cache Stampede Prevention* (PVLDB 8(8), 2015)
- Breslau, Cao, Fan, Phillips, Shenker. *Web Caching and Zipf-like Distributions: Evidence and Implications* (INFOCOM 1999)
- RFC 9111. *HTTP Caching* (2022); RFC 5861. *HTTP Cache-Control Extensions for Stale Content* (2010)
- Redis Documentation. *Key eviction*; Caffeine Documentation. *Population*
- Go. `golang.org/x/sync/singleflight` (pkg.go.dev)

Список источников (2/2)

- RFC 9110. *HTTP Semantics* (2022), §15.3.3 «202 Accepted»; RFC 896. *Congestion Control in IP/TCP Internetworks* (Nagle, 1984)
- Microsoft Azure Architecture Center. *Asynchronous Request-Reply pattern*
- Stripe API Reference. *Idempotent requests*
- Richardson. *Transactional outbox* (microservices.io)
- Apache Kafka Documentation. *Producer configs* (`batch.size`, `linger.ms`)
- Kleppmann. *Designing Data-Intensive Applications* (O'Reilly, 2017): гл. 6 «Секционирование»
- Dean, Barroso. *The Tail at Scale* (Communications of the ACM, 2013)
- Figma Engineering. *How Figma's databases team lived to tell the scale* (2024)
- Zhang (OpenAI). *Scaling PostgreSQL to power 800 million ChatGPT users* (22 января 2026)
- PostgreSQL Documentation. *Appendix K. PostgreSQL Limits*
- Fowler. *MicroservicePremium и MonolithFirst* (martinfowler.com, 2015)
- Kolny. *Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%* (Prime Video Tech, 2023)
- Knuth. *Structured Programming with go to Statements* (ACM Computing Surveys 6(4), 1974)
- Dean. *Latency Numbers Every Programmer Should Know* (~2012)

Обратная связь

Анонимная форма